Appunti del corso di
Intelligenza artificale

UniVR - Dipartimento di Informatica

Primo semestre - A.A. 2025/26

Mattia Nicolis Matteo Drago

Indice

1 Introduzione **1**
1.1 Tipologie di intelligenza artificiale . . . 2
1.1.1 Agenti autonomi . . . 2
1.1.2 Data analysis . . . 2
1.1.3 Machine Learning . . . 2
1.1.4 Time series analysis . . . 2
1.1.5 Agenti intelligenti . . . 3
1.1.6 Markov Decision Process (MDP) . . . 3
1.1.7 Generative AI . . . 3

2 Agenti intelligenti **4**
2.1 Misure di prestazione . . . 6
2.1.1 Razionalità . . . 6
2.2 Specificare un ambiente (PEAS) . . . 6

3 Risolvere i problemi con la ricerca **7**
3.1 Agenti risolutori di problemi . . . 7
3.2 Algoritmi di ricerca . . . 7
3.2.1 Tree search . . . 8
3.3 Strategie di ricerca non informata . . . 9
3.3.1 Ricerca in ampiezza . . . 9
3.3.2 Ricerca a costo minimo . . . 10
3.3.3 Ricerca in profondità . . . 11
3.3.4 Ricerca a profondità limitata . . . 11
3.4 Strategie di ricerca informata . . . 12
3.4.1 Ricerca best-first greedy . . . 13
3.4.2 RIcerca A* . . . 14
3.5 Funzioni euristiche . . . 15
3.5.1 Effetto dell'acuratezza dell'euristica sulle prestazioni . . . 15
3.5.2 Generare uristiche da problemi rilassati . . . 16

4 Ricerca in ambienti complessi **17**
4.1 Ricerca locale . . . 17
4.1.1 Ricerca hill climbing . . . 17
4.1.2 Simulated annealing . . . 19
4.1.3 Ricerca local beam . . . 19

4.1.4 Algoritmi evolutivi 19
4.2 Ricerca locale in spazi continui 19

5 Problemi di soddisfacimento dei vincoli **20**
5.1 Definizione dei problemi di soddisfacimento dei vincoli 20
5.1.1 Varianti del formalismo CSP 22
5.2 Programmazione dei vincoli: inferenza nei CSP 22
5.2.1 Consistenza di nodo 22
5.2.2 Consistenza d'arco 22
5.2.3 Consistenza di cammino 23
5.2.4 K-consistenza 23
5.2.5 Vincoli globali 23
5.3 Ricerca con backtracking per CSP 23

1

Introduzione

Il compito dell'**intelligenza artificale** (IA) non è solo quello di *comprendere*, ma anche di *costruire* entità intelligenti.

Definizione: entià intelligenti

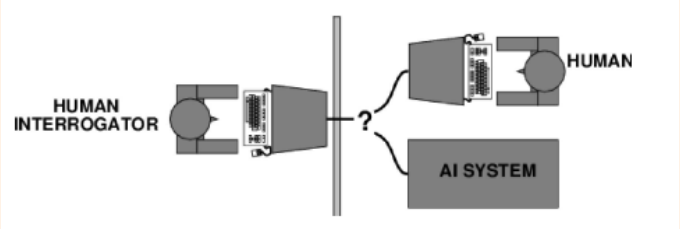
Macchine in grado di comprendere come **agire in modo efficace e sicuro** in un'ampia varietà di situazioni nuove.

In passato, gli studiosi hanno proposto diverse versioni di IA:

- **IA in termini di *fedeltà alla prestazione umana***: una scienza empirica correlata alla psicologia, richiedendo osservazioni riguardo al comportamento umano e ai procesi di pensiero
- **IA in termini di *razionalità***: sfrutta una combinazione di matematica e ingegneria e si collega alla statistca, alla teoria del controllo e all'economia

Test di Turing

Il **test di Turing**, proposto da Alan Turing nel 1950, mira a comprendere se una macchina è in grado di pensare.
Un computer supera il test, se un esaminatore umano, dopo aver posto alcune domande in forma scritta, non è in grado di distinguere se le risposte provenfono da una persona o da una macchina.



Per superare il test, il computer deve possedere le seguenti capacità:

- **interpretazione del linguaggio naaturale** per comunicare con successo nel linguaggio umano
- **rappresentazione della conoscenzza** per memorizzare ciò che *conosce* o *apprende*

- **ragionamento automatico** per rispondere alle domande e trarre nuove conclusioni
- **apprendimento autatico** (*machine learning*) per adattarsi a nuove circostanze, individuare ed esplorare schemi

Esiste, inoltre un cosidetto **test di Turing totale**, che richiede l'interazione con oggetti e persone nel mondo reale.
Per superare qust'ultimo tipo di test, un robot deve anche possedere:

- **visione artificiale** e **riconoscimento vocale** per percepire il mondo
- **robotica** èer manipolare gli oggetti e muoversi fisicamente

1.1 Tipologie di intelligenza artificiale

1.1.1 Agenti autonomi

Sono sistemi che percepiscono l'ambiente e agiscono in modo autonomo per raggiungere obiettivi specifici.

1.1.2 Data analysis

Consiste nell'utilizzo di algoritmi per analizzare grandi quantità di dati ed estrarre informazioni utili e correlazioni complesse.

1.1.3 Machine Learning

È lo sviluppo di algoritmi che permettono ai modelli di apprendere dai dati di esempio e migliorare le proprie prestazioni nel tempo, senza essere esplicitamente programmati.

Un esempio è il riconoscimento di immagini.

L'apprendimento automatico si divide in tre categorie principali:

- **unsupervised learning**: l modello viene addestrato su dati non etichettati, con l'obiettivo di scoprire strutture nascoste o pattern nei dati senza risposte predefinite.
- **supervised learning**: l modello viene addestrato su dati etichettati, dove ogni input è associato a una risposta corretta. L'obiettivo è imparare a mappare correttamente gli input alle risposte
- **reinforced learning**: il modello apprende attraverso interazioni con l'ambiente, ricevendo ricompense o penalità in base alle azioni compiute. L'obiettivo è massimizzare la ricompensa totale nel tempo

1.1.4 Time series analysis

L'analisi delle serie temporali è un'area dell'apprendimento automatico che si concentra sull'analisi di dati raccolti nel tempo.

Le serie temporali sono sequenze di dati misurati a intervalli regolari, come la temperatura giornaliera, i prezzi delle azioni o i dati di vendita mensili.

L'obiettivo è identificare pattern, tendenze e stagionalità per effettuare previsioni future.

Gli approcci comuni includono:

- **riconoscimento di anomalie e cause**: identificazione di eventi che si discostano dal comportamento normale, potenzialmente indicativi di errori o problemi.
- **generative transformers**: modelli in grado di predire il prossimo elemento in una sequenza di dati (ad esempio la parola successiva in una frase o il pixel successivo in un'immagine), utilizzando il concetto di **attenzione** per pesare l'importanza delle diverse parti della sequenza di input.

1.1.5 Agenti intelligenti

Un agente intelligente è un sistema che percepisce l'ambiente circostante attraverso sensori e agisce su di esso per raggiungere un obiettivo specifico.

Gli elementi chiave di un agente intelligente sono:

- **performance measure**: misura il successo dell'agente nel raggiungere i propri obiettivi
- **rationality**: l'agente deve agire in modo da massimizzare la performance attesa

1.1.6 Markov Decision Process (MDP)

Un MDP è un modello matematico utilizzato per rappresentare problemi di decisione sequenziali. I suoi elementi principali sono:

- **state**: rappresenta l'ambiente in un determinato momento
- **actions**: nsieme delle azioni che l'agente può intraprendere
- **transition model**: descrive l'effetto delle azioni sull'ambiente (può essere parzialmente incognito)
$$T : (state, action) \rightarrow next_state$$
- **reward**: vvalore immediato associato all'esecuzione di un'azione
$$R : (state, action, next_state) \rightarrow real_number$$
- **Policy**: strategia con cui l'agente decide quale azione intraprendere in ogni stato per massimizzare la ricompensa totale attesa nel tempo
$$\pi : (state) \rightarrow action$$

1.1.7 Generative AI

L'intelligenza artificiale generativa comprende una classe di modelli in grado di creare nuovi contenuti — testo, immagini, musica o video — a partire da dati di addestramento.

Questi modelli possiedono miliardi di parametri e sono pre-addestrati su enormi quantità di dati.

In sostanza, "predicono il futuro" basandosi sui dati su cui sono stati **addestrati** e su un **prompt** (input dell'utente).

2

Agenti intelligenti

Un **agente** è una qualsiasi *entità* che può essere visto come un **sistema** che *percepisce* il proprio ambiente **attraverso** dei **sensori** e *agisce* su di esso mediante **attuatori**.

Un agente umano possiede come **sensori** gli *occhi* e *altri organi*, e può utilizzare come **atturatori** *mani*, *gambe*, *corde vocali* ecc.

Un agente robotico può, invece, disporre di *telecamere* e *telemetri a infrarosso* (es. sensori e diversi motori) come**attuatori**.

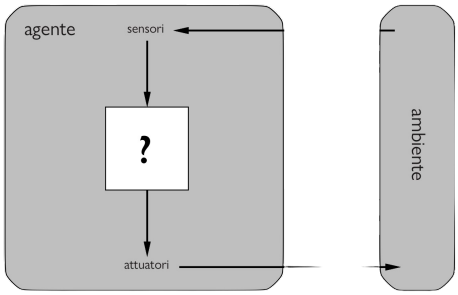


Figura 2.1: agente razionale

Definizione: agente razionale (formale)

Per ogni possibile seuqenza di percezioni, un agente razionale dovrebbe scegliere un'azione che **massimizzi** il valore atteso della sua *performance measure*, date le informazioni fornite dalla suquenza e da ogni ulteriore conoscenza posseduta dall'agente.

In termini matematici, il comportamento di un agente è descritto dalla **funzione agente**, che descrive la corrispondenza tra una qualsiasi sequenza percettiva e una specifica azione:

$$f : \mathcal{P}* \rightarrow \mathcal{A}$$

E' importante fare una distinzione tra due concetti distinti:

- **funzione agente**: descrizione matematica astratta
- **programma agente**: implementazione concreta della funzione agente, in esecuzione all'interno di un sistema fisico

Calcolo (...da rinominare)

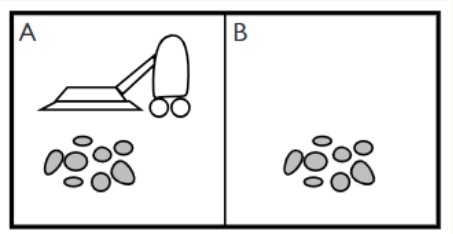
Se un agente ha $|P|$ possibili percezioni, quante voci avrà la funzione dopo T passi temporali?

$$\sum_{t=1}^{T} |P|^t$$

L'obiettivo dell'IA è di progettare piccoli programmi agente per rappresentare grandi funzioni agenti.

Esempio: agente aspirapolvere robotizzato

Consideriamo un agente aspirapolvere robotizzato in un mondo costituito da riquadri che possono essere sporchi o puliti.



L'agente aspirapolvere percepisce in quale riquadro si trova e se tale riquadro è sporco. Parte nel riquadro A e le azioni che può compiere sono:

- *muoversi a destra*
- *muoversi a sinistra*
- *aspirare lo sporco*
- *non fare nulla*

Una funzione agente molto semplice è: "*se il riquadro corrente è sporco, aspira, altrimenti muoviti nell'altro riquadro*".

Sequenza percettiva	Azione
[A, pulito]	azione
[A, sporco]	destra
[B, pulito]	aspira
[B, sporco]	sinistra
[A, pulito], [A, pulito]	destra
[A, pulito], [A, sporco]	aspira
⋮	
[A, pulito], [A, pulito], [A, pulito]	destra
[A, pulito], [A, pulito], [A, sporco]	aspira
⋮	

Codice: programma agente - agente aspirapolvere

```
function Reflex-Vacuum-Agent([location, status]) returns actions
  if status = dirty then return suck
```

```
        else if location = A then return right
        else if location = B the return left
```

Qual'è il modo più corretto di progettare la tabella? Che cosa rende un agente buono o cattivo, intelligente o stupido?

Abbiamo bisogno di una **performance measure**.

2.1 Misure di prestazione

L'intelligenza artificiale è fedele al concetto di **consequenzialismo**: valutare il comportamento di un agente considerandone le *conseguenze*.

Quando un agente viene iscritto in un ambiente, genera una sequenza di azioni in base alle percezioni che riceve.

Questa sequenza di azioni porta l'ambiente ad attraversare una sequenza di stati: se tale sequenza è *desiderabile*, significa che l'agente s è comportato bene.

Questa nozione di desiderabilità è catturata da **misura di prestazione** (*performance measure*) che valuta una seuqenza di stati dell'ambiente.

2.1.1 Razionalità

Definizione: agente razionale

Per ogni possibile sequenza di percezioni, un agente dovrebbe scegliere un'azione che massimizzi il valore atteso della sua misura di prestazione, date le informazioni fornite dalla sequenza percettiva e da ogni ulteriore conoscenza dell'agente.

Bisogna, però, distinguere un paio di concetti:

- **razionalità** $\neq$ **onniscenza**: le percezioni potrebbero non fornire tutte le informazioni rilevanti
- **razionalità** $\neq$ **chiaroveggente**: i risultati dell'azione potrebbero non essere quelli previsti
- **razionalità** $\neq$ **riuscito**

Razionale vuol dire *esplorazione*, *apprendimento*, *autonomia*.

2.2 Specificare un ambiente (PEAS)

Per progettare un agente intelligente bisogna definire l'ambiente in cui si opera:

- **performance measure**: come viene valutato l successo dell'agente
- **enviroment**: contesto in cui opera l'agente
- **actuators**: mezzi attraverso cui l'agente agisce sull'ambiente
- **sensors**: mezzi attraverso cui l'agente percepisce l'ambiente

3

Risolvere i problemi con la ricerca

Quando l'azione giusta da compiere non è subito evidente, un agente può avere la necessità di *guardare avanti*, cioè considerare una sequenza di azioni che formano un cammino che porterà a uno **stato obiettivo**.

Questo agente è chiamato **agente risolutore di problemi** e il processo computazionale che effettua è una *ricerca*.

Questo tipo di agente utilizza *rappresentazioni atomiche* in cui gli stati del mondo sono considerate come entità prive di una struttura interna visibile agli algoritmi per la risoluzione dei problemi.

GLi agenti che utilizzano *rappresentazioni di stati finiti* o *strutture* sono chiamati **agenti pianificatori**.

3.1 Agenti risolutori di problemi

Gli **agenti risolutori di problemi** sono una forma ristretta di agenti che formulano un problema e un obiettivo partendo da uno stato, ignorando le percezioni.

Questo si chiama **offline problem solving** perchè l'agente ha completa conoscenza dell'ambiente.

3.2 Algoritmi di ricerca

Definizione: algoritmo di ricerca

Un **algoritmo di ricerca** riceve in *input* un roblema e restituisce come *ouput* una soluzione o un fallimento.

3.2.1 Tree search

Consideriamo, quindi, algoritmi che sovrappongono un **albero di ricerca** al *grafo dello spazio degli stati*, formanod vari cammini a partire dallo stato iniziale e cercando di trovarne uno che raggiunge uno stato obiettivo.

Un albero di ricerca è costituito da:

- **nodo**: corrisponde a uno stato nello spazio degli stati
- **ramo**: corrisponde a un'azione

La radice dell'albero corrisponde allo sttao iniziale del problema.

Nell'albero di ricerca possono esserci più cammini per raggiungere qualsiasi stato, ma per ogni nodo dell'albero c'è un cammino univoco per tornare alla radice.

Codice: Tree search

```
function Tree-Search(problem, strategy) returns solution or failure
  loop do
    if no candidates for expansion the return failure
      choose a leaf node for expansion according to strategy
    if node contains a goal state the return solution
    else add successor nodes to the search tree (expansion)
  end
```

Possiamo **espandere** il nodo, considerando le *azioni* disponibili per quello stato, usando la funzioe *risultato* per vedere dove portano tali azioni e **generare** un nuovo nodo (*nodo figlio sucessore*) per ognuno degli stati risultanti.
Ogni nodo figlio ha un **nodo padre**.

Chiamiamo l'insieme di questi nodi **forntiera** dell'albero di ricerca, e diciamo che ogni stato per cui vi è un nodo generato è stato *raggiunto*.

Codice: Tree search - generale

```
function Tree-search(problem, frontier) returns solution or failure
  forntier <- insert(make-node(problem.initilaState))

  while not isEmpty(frontier) do
    node <- pop(frontier)
    if problem.goalTest(node.state) then return node
      forntier <- insertAll(expand(node, problem))
  end loop

  return failure
```

La forntiera *separa* due regioni del grafo dello spazio degli stati:

- **regione interna**: ogni stato è stat espanso
- **regione esterna**: stati che non sono stati ancora raggiunti

3.3 Strategie di ricerca non informata

Un **algoritmo di ricerca non informata** *non* riceve alcuna informazione su quanto uno stato sia vicino all'obiettivo.

Esempio: ricerca non informata

Consideriamo una persona che si trova ad Arad con l'obiettivo di raggiungere Bucarest.

Una persona non informata, priva di conoscenza della geografia romena, non ha modo di sapere se sia meglio come primo passo andare a Zerind o Sibiu

3.3.1 Ricerca in ampiezza

La **ricerca in ampiezza** è una strategia di ricerca che si può applicare quando le azioni hanno le stesso costo: si espande per primo il *nodo radice*, poi tutti i *sucessori*, poi i *sucessori* di quest'ultimi e così via.

Si potrebbe implementare con la *funzione di valutazione* $f(n)$ uguale alla profondità del nodo (numero di azioni per raggiungerlo), ma con alcuni trucchi si può ottenere una maggiore efficienza.
Una *coda FIFO* (First In First Out) risulterà più veloce di una *coda con priorità* e ci fornisce l'ordine corretto dei nodi: i nuovi nodi vanno in fondo alla coda e quelli vecchi, a profondità minore, vengono espansi per primi.
Inoltre, *raggiunti* può essere un insieme di stati, anzichè una corrispondenza da stati a nodi, perchè una volta raggiunto uno stato, non si può più trovare un cammino migliore per raggiungerlo.
Questo significa che si puà effettuare in **test obiettivo anticipato**, controllando se un nodo è una soluzione non appena non viene *generato*, invece di effettuare il **test obiettivo a posteriori**, aspettando che un nodo sia estratto dalla coda.

La ricerca in ampiezza **trova sempre una soluzione** con un numero minimo di azioni, perchè quando genera nodi di profondità d, ha già generato tutti i nodi di profondità $d-1$, perciò se uno di essi fosse una soluzione, sarebbe stato trovato.
Questo significa che è *ottimale rispetto al costo* per i problemi in cui tutte le azioni hanno lo stesso costo.

In *termini di tempo* e *spazio*, supponiamo di effettuare una ricerca in un albero uniforme dove ogni stato ha b sucessori: la radice dell'albero di ricerca genera b nodi, ognuno dei quali genera altri b nodi, per un totale di b^2 al secondo livello.
Di conseguenza, il numero totale di nodi generati è:

$$1 + b + b^2 + b^3 + \cdots + b^d = O(b^d)$$

Codice: algoritmo BFS

```
function BFS(problem) returns solution o failure
  node <- node with state=proble.inistualState, pathCost = 0

  if problem.goalTest(node.State) the return node
```

```
          explored <- emptyTest
          forntier <- FIFO queue with node as the only element

        loop do
          if forntier is empty the return failure
            node <- pop(forntier)
            explored.add(node.state)

            for each action in problem.actions(node.state) do
              child <- child-node(problem, node, action)

              if child.state is not explored or frontier then
                if problem.goalTest(child.state) the return child
              end if
            end for
        end loop
```

Tutti i nodi rimangono in memoria, perciò la complesità temporale e spaziale sono entrambe $O(b^d)$, quindi, esponenziali.

Nella ricerca in ampiezza, i requisiti di memoria rappresentano un problema più importante rispetto al tempo di esecuzione.
Tuttavia, il tempo rimane comunque un fattore importante.

N.B.

In generale, i problemi di ricerca con complesità esponenziale non possono essere risolti mediante ricerche non informate, se non nelle istanze più piccole.

3.3.2 Ricerca a costo minimo

Quando le azioni hanno costi diversi, si utilizza una BFS in cui la *funzione di valutazione* è il costo del cammino dalla radice al nodo corrente; si chiama **ricerca a costo uniforme** o **algoritmo di Dijkstra**.

La complessità della ricerca a costo uniforme è caratterizzata in termini di:

- C* (costo dela soluzione ottima)
- $\epsilon > 0$ (limite inferiore imposto al costo di ogni azione)

Di conseguenza, la complessità sarà:

- *caso peggiore* (costi diversi): complessità temporale e spazilae $= O(b^{1+\lfloor C^*/\epsilon \rfloor})$
- *caso migliore* (costi uniformi): complessità temporale e spaziale $= O(b^{d+1})$

La ricerca a costo uniforme è completa e ottima *rispetto al costo* perchè la prima soluzione che trova avrà un costo basso almeno quanto quello di ogni altro nodo sulla frontiera.

Questa ricerca esamina tutti i cammini in ordine di costo crescente, quindi, non rientra mai in un cammino infinito (assumendo che tutti i costi delle azioni siano $> \epsilon > 0$)

3.3.3 Ricerca in profondità

La **ricerca in profondità** espande sempre per primo il nodo a *profondità maggiore* nella frontiera.

Potrebbe essere implementata con una chiamata di ricerca BFS in cui la funzione di valutazione f è l'opposto (negativo) dela profondità.
Generalmente viene implementata come ricerca ad albero che non mantiene una tabella egli stati raggiunti.

La ricerca in profondità non è ottima rispetto al costo, infatti, restituisce sempre la prima soluzione che trova anche se non è la meno costosa.

Per spazi degli **stati finiti** che sono alberi, la ricerca in profondità è efficiente e completa; per spazi degli **stati aciclici** potrebbe arrivare a espandere lo stesso stato più volte attraverso cammini diversi, ma (alla fine) esplorerà sistematicamente l'intero spazio.

In spazi degli **stati ciclici**, la ricerca in profondità può bloccarsi in un ciclo infinito, perciò alcune implementanzioni controllano ciascun nodo nuovo per verificare la presenza di cicli.
Infine, in spazi degli **stati infiniti** la ricerca in profondità non è sistematica: può entrare in un cammino infinito, anche se non vi sono cicli.

Quindi, la ricerca in profondità è incompleta.

Per i problemi in cui è praticabile un ricerca ad albero, la ricerca in profondità ha esigenze di memoria molto più ridotte.

Non viene mantenuta una tabella dei nodi raggiunti, e la frontiera è molto più piccola: mentre nella ricerca in ampiezza la frontiera può essere vista come la superficie di una sfera in continua espansione, nella ricerca in profondità la frontiera è solatanto un raggio della sfera.

Per uno spazio degli stati finito e a forma di albero una ricerca ad albero in profondità richiede un tempo proporzionale al numero degli stati e ha una complessità di memoria solo $O(bm)$, dove b è il fattore di ramificazione e m è la massima profondità dell'albero.

3.3.4 Ricerca a profondità limitata

Per evitare che la ricerca in profondità si perda in un cammino infinito possiamo usare la **ricerca a profondità limitata**, in cui forniamo un limite di profondità ℓ e consideriamo tutti i nodi a profondità ℓ come se non avessero sucessori.
$\Rightarrow$ *complessità temporale* $= O(b^\ell)$
$\Rightarrow$ *complessità spaziale* $= O(b\ell)$
Se si sceglie male ℓ l'algoritmo non riuscirà a trovare la soluzione, per cui è ancora **incompleto**.

Poichè la ricerca in profondità è una ricerca ad albero, non possiamo evitare che perda tempo su cammini rindondanti in generale, ma possiamo eliminare i cammini ciclici al costo di un pò di tempo di calcolo.
Se esaminiamo soltanto pochi collegamenti risalendo nella catena di parentela, riusciamo a catturare la maggior parte dei cicli; qulli più lunghi sono rimossi grazie al limite sulla profondità.
A volte si può scegliere un buon limite di profondità basandosi sulla conoscenza del problema.

La **ricerca ad approfondimento iterativo** risolve il problema di trovare un buon valore di ℓ provando tutti i valori: prima 0, poi 1, poi 2 e così via, fino a quando si trova una solzuione, ppoure la ricerca a profondità limitata restituisce il valore *fallimento* anzichè il valore *soglia*.

Codice: ricerca ad approfondimento iterativo

```
function IDS(problem) returns solution
  inputs: problem

  for depth <- 0 to $\infty$ do
    result <- DLS(problem, depth)

    if reuslt != then return result
  end
```

L'approfondimento iterativo riunisce molti vantaggi offerti dalla ricerca in profondità e da quella in ampiezza:
$\Rightarrow$ dalla **ricerca in profondità**: requisiti di memoria modesti

- se esiste una soluzione $\rightarrow O(bd)$
- se non esiste una soluzione (spazi degli stati finiti) $\rightarrow O(bm)$

$\Rightarrow$ dalla **ricerca in ampiezza**: otima per problemi in cui tutte le azioni hanno lo stesso costo, ed è completa su spazi degli stati aciclici finiti, o su qualsiasi spazio degli stati finito quando controlliamo i nodi per la presenza di cicli risalendo l'intero cammino.

In generale, la *complessità temporale* vale $O(b^d)$ se esiste suluzione e $O(b^m)$ se non esiste la soluzione.

Criterio	In ampiezza	A costo uniforme	In profondità	A profondità limitata	Ad approfondimento iterativo
Completa?	Sì	Sì	No	No	Sì
Ottima rispetto al costo?	Sì	Sì	No	No	Sì
Tempo	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$
Spazio	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b^\ell)$	$O(b^d)$

3.4 Strategie di ricerca informata

La **ricerca informata** sfrutta la conoscenza specifica del dominio applicativo per fornire suggerimenti su dove si porebbe trovare l'obiettivo.
I suggerimenti hanno la stessa forma di una *funzione euristica* ($h(n)$).

Definizione: funzione euristica $h(n)$

Costo stimato del cammino meno costo dallo stato del nodo n a uno stato obiettivo.

3.4.1 Ricerca best-first greedy

La **ricerca best-first greedy** è una forma di ricerca BFS che espande prima il nodo con il valore più basso di $h(n)$, quello più vicino all'obiettivo, sulla base del fatto che è probabile che questo porti rapidamente a una solzuione.
$\Rightarrow$ *funzione valutazione* $f(n) = h(n)$

Esempio: viaggio in Romania

Consideriamo il problema dell'itinerario in Romania usando come uristica la **distanza in linea d'aria** (h_{DLA}).
Se l'obiettivo è Bucarest, dovremo conoscere tutte le distanze in linea d'aria verso quella città.

Arad	366	**Mehadia**	241
Bucarest	0	**Neamt**	234
Craicova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

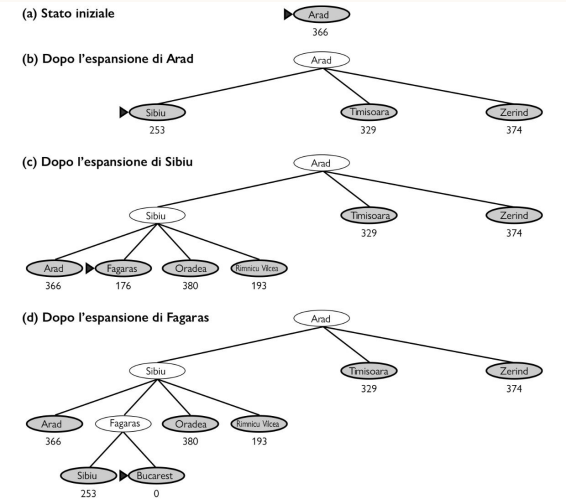


Figura 3.1: Algoritmo BFS greedy

Il primo nodo espanso da Arad sarà Sibiu, perchè l'uristica afferma che è più vicino s Bucarest di Zerind e Timisoara.
Il nodo sucessivo sarà Fagras, perchè ora è il più vicino secondo l'euristica.
Fagras a sua volta generà Bucarest, che è l'obiettivo.

Tuttavia, la soluzione trovata non ha un costo ottimo, infatti il cammino che passa da Sibiu-Fagras è di 32 miglia più lungo di quello che attraversa Rimnicu Vilcea-Pitesti.

La ricerca BFS greedy su grafo è competa negli spazi degli stati finiti, ma non in quelli infiniti.

$\Rightarrow$ *complesità temporale* e *spaziale* $= O(|V|)$ [caso peggiore]
$\Rightarrow$ *complessità temporale* e *spaziale* $= O(bm)$ [con buona euristica]

3.4.2 RIcerca A*

La forma più diffusa di algoritmo di ricerca informata è la **ricerca A***, una ricerca BFS che utilizza come *funzione valutazione*:

$$f(n) = h(n) + g(n)$$

h(n) : costo stimato dal cammino più breve da n a uno stato obiettivo
g(n) : costo stimato dal cammino iniziale al nodo n
f(n) : costo stimato del cammino migliore che continua da n fino a un obiettivo

Esempio: viaggio in Romania

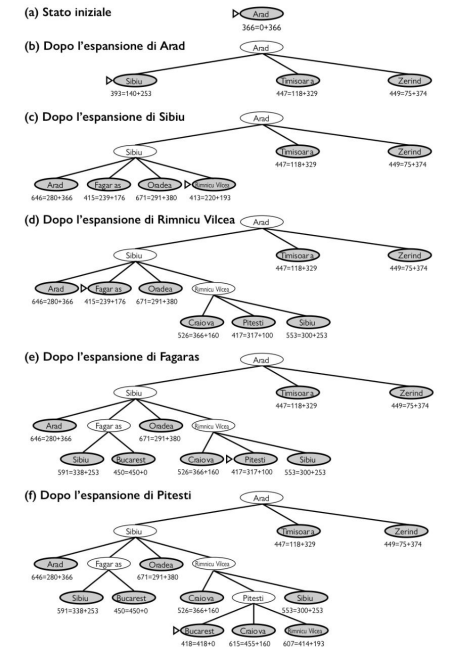


Figura 3.2: Algoritmo A*

Notiamo che Bucarest appare prima sulla frontiera al passo (e), ma non è selezionata per l'espansione (non individuata come suluzione) perchè con $f = 450$ non è il nodo di costo minimo sulla frontiera, che, invece, è Pitesti con $f = 417$.
In altre parole, significa che potrebbe esistere una soluzione tramite Pitesti con costo di 417, per cui l'algoritmo non si ferma a una solzuione di costo 450.
Al passo (f), un diverso cammino verso Bucarest è il nodo di costo minimo (f=418), per cui viene selezionato e individuato come soluzione ottima.

La ricerca A* è completa.
Il fatto che sia meno ottimale rispetto al costo dipende da alcune proprietà dell'uristica.

Una proprietà fondamentale è l'**ammissibilità**.

Definizione: euristica ammissibile

Un'**euristica ammissibile** è tale se non sovrastima mai il costo per raggiungere un obiettivo.

Con un'euristica ammissibile la ricerca A* è ottima *rispetto al costo*.

Definizione: euristica conistente

Un'euristica $h(n)$ si dice **consistente** (o monotona) se, per ogni nod n e ogni sucessore n' generato mediante un'azione a, il costo stimato di raggiungere l'obietivo da n è minore o uguale al costo per raggiungere n' per il costo stimato di raggiungere l'obiettivo da n':

$$h(n) \leq c(n, a, n') + h(n')$$

Questa è una forma della **disugualianza triangolare**, per cui ogni lato di un triangolo deve essere minore o uguale alla somma degli altri due lati.

Ogni **euristica conistente è anche ammissibile** (non viceversa), perciò la ricerca A* con un'euristica consistente è ottima *rispetto al costo*.

Con un'euristica consistente, la prima volta che raggiungiamo uno stato sarà su un cammino ottimo, perciò non dovremo mai raggiungere di nuovo uno stato alla frontiera, nè dovremm mai modificare un elemento in *raggiunti*.

3.5 Funzioni euristiche

3.5.1 Effetto dell'acuratezza dell'euristica sulle prestazioni

Un modo di caratterizzare la qualità di un'euristica è il **fattore di ramificazione efettivo** (b*).
Se il numero totale di nodi generati da A* per un particolare problema è *N*, e la profondità è *d*, allora b* è il fattore di ramificazione che un albero uniforme in profondità *d* dovrebbe avere per contenere *N + 1* nodi:

$$N + 1 = 1 + b* + (b*)^2 + \cdots + (b*)^d$$

Esempio:

Se A* trova una soluzione a profondità 5 usando 52 nodi, il fattore di ramificazione effettivo è 1.92

Questo fattore può cambiare da un'istanza del problema all'altra, ma solitamente per un dominio del problema specifico ha un valore abbastanza costante su tutte le istante di problemi

non banali.
DI conseguenza, misurazioni sperimentali di b* su un numero abbastanza piccolo di problemi può già fornire una buona idea dell'utilità generale dell'euristica.

Un'eristica ben progettata dovrebbe avere un valore di b* vicino a 1, permettendo così la soluzione di problemi abbastanza grandi a un costo computazionale ragionevoli.

3.5.2 Generare uristiche da problemi rilassati

⇒ *Com'è possibile inventare un'eristica come h_2 ?*

h_1 e h_2 sono stime della lunghezza del cammino minimo da uno stato a quello obiettivo, ma sono anche lunghezze di cammino perfettamente accurate per versioni semplificate del rompicapo.
Se le regole del problema fossero cambiate in modo che un tassello potesse muoversi ovunque, invece che solo nello spazio vuoto, il costo minimo per raggiungere l'obiettivo sarebbe esattamente h_1.
In modo analogo, se un tassello potesse muoversi di uno sspazio in ogni direzione, anche se la posizione fosse già occupata da un altro tassello, il costo minimo per raggiungere l'obiettivo sarebbe esattamente h_2.

Un problema con meno vincoli sulle azioni è detto un **problema rilassato**.

Il grafo dello spazio degli stati per il problema rilassato è un superparagrafo di quello dello spazio degli stati originale, perchè la rimozione dei vincoi crea nuovi archi nel grafo.

4

Ricerca in ambienti complessi

4.1 Ricerca locale

Gli **algoritmi di ricerca locale** operano a partire da uno *stato iniziale* e procedendo verso gli *stati adiacenti* senza tenere traccia dei cammini, nè degli stati già raggiunti.

Questo significa che **non sono sistematici**: potrebbero anche non esplorare mai una porzione dello spazio degli stati dove in effetti risiede una soluzione.

Hanno, però, due **vantaggi**:

- usano pochissima memoria
- spesso trovano una soluzione ragionevole in spazi degli stati grandi o infiniti

Gli algoritmi di ricerca di ricerca locale possono anche risolvere *problemi di ottimizzazione*, in cui lo scopo è trovare lo stato migliore secondo una *funzione obiettivo*.

4.1.1 Ricerca hill climbing

L'agoritmo di ricerca **hill climbing** tiene tracica di un solo stato corrente e a ogni iterazione passa allo stato vicino con valore più alto, cioè punta nella direzione che presenta l'*ascesca più rapida*, senza guardare oltre gli stati immediatamente vicini a quello corrente.

L'algoritmo hill climbing viene talvolta chiamato **ricerca locale greedy** perchè sceglie uno stato vicino "buono" senza pensare a come andrà avanti.

Codice: algoritmo hill climbing

```
function Hill-Climbing(problema) returns state that is a locale
    maximum
  current <- problem.InitialState

  loop do
    neighbour <- highest valued sucessor of current

    if neighbour.Value <= current.Value the return current
    current <- neighbour
  end
```

Esempio: problema delle n-regine

Questo problema consiste nel posizionare n regine su una scacchiera $n \times n$ in modo che nessuna regina possa attaccare un'altra (no due regine su stessa riga/colonna/diadonale).

Consideriamo per esempio il caso con $n = 3$.
Ad ogni passo, spostiamo una regina in modo da preservare le condizioni iniziali e ridurre il numero di confilitti.

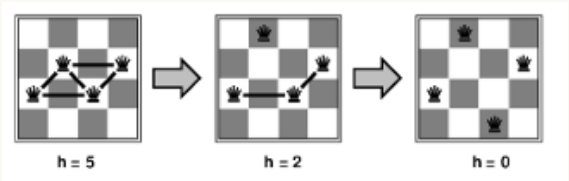


Figura 4.1: problema n-regine

L'algoritmo hill climbing può procedere molot rapidamente verso la soluzione, perchè di solito è abbastanza facile uno stato sfavorevole.

L'hill climbing può procedere molto rapidamente verso la soluzione, perchè di solito è abbastanza facile migliorare uno stato sfavorevole.
Spesso, però, l'hill climbing rimane bloccato per le seguenti ragioni:

- **massimi locali**: un massimo locale è un picco più alto degli stati vicini, ma inferiore al massimo globale.
  Gli algoritmi hill climbing che raggiungono la vicinanza di un massimo locale saranno attirati verso il picco, ma rimarranno bloccati lì senza poter andare altrove.
- **creste**: le creste danno origine a una sequenza di massimi locali molto difficili da esplorare da parte degli algoritmi greedy
- **plateau**: un plateau è un'area piatta del panorama dello spazio degli stati.
  Può essere un massimo locale piatto, da cui non è possibile fare ulteriori progressi, oppure una **spalla**, da cui si potrà salire ulteriormente (hill climbing potrebbe perdersi sul plateau)

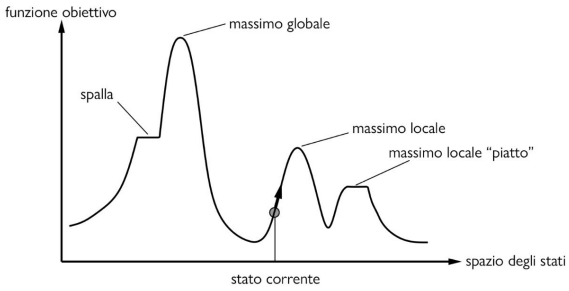


Figura 4.2: intoppi hill climbing

In ognuno di questi casi, l'algoritmo raggiunge un punto dal quale non riesce a compiere ulteriori progressi.

⇒ *Come possiamo risolvere un maggior numero di istanze?*

Un modo è quello di proseguire nella ricerca un volta raggiunto un plateau, consentendo una **mossa laterale** nella speranza che il plateau sia in realtà una spalla.
Nel caso in cui fossimo realmente su un massimo locale piatto, con questo approcio finiremmo per vagare sul plateau per sempre.

4.1.2 Simulated annealing

Codice: algoritmo simulated annealing

```
function Simulated-Annealing(problem, schedule) returns
    solution
  current <- problem.statoIniziale
```

4.1.3 Ricerca local beam

4.1.4 Algoritmi evolutivi

4.2 Ricerca locale in spazi continui

5

Problemi di soddisfacimento dei vincoli

5.1 Definizione dei problemi di soddisfacimento dei vincoli

Un **problema di soddisfacimento dei vincoli** è costituito da tre componenti $\mathcal{X}$, $\mathcal{D}$, $\mathcal{C}$.

$\mathcal{X}$: insieme di *variabili* $[X_1, \ldots, X_n]$
$\mathcal{D}$: insieme dei *domini* $[D_1, \ldots, D_n]$ (uno per ogni variabile)
$\mathcal{C}$: insieme di *vincoli* (specificano combinazioni di valori ammissi)

Un dominio $\mathcal{D}_i$ è costituito da un insieme di valori ammessi $\{v_1, \ldots, v_n\}$, per la variabile $\mathcal{X}_i$.

Esempio: valori ammissi per un dominio $\mathcal{D}_i$

Una variabile booleana avrebbe il dominio $\{vero, falso\}$.

Variabili diverse possono avere domini differenti e di dimensioni diverse.

Ogni vincolo $\mathcal{C}_i$ è costituito da una coppia $\langle$ ambito, ref $\rangle$:

ambito : *tupla di variabili* che partecipano al vincolo
ref : *relazione* che definisce i valori che tali variabili possono assumere

Una relazione può essere rapprentata come:

- **insieme esplicito** di tutte le tuple di valori che soddisfano il vincolo
- **funzione** che è in grado di determinare se una tupla è membro della tupla

Esempio: relazione

Consideriamo due variabili $\mathcal{X}_1$ e $\mathcal{X}_2$, entrambe con dominio $\{1, 2, 3\}$.
Il vincolo che afferma che $\mathcal{X}_1$ deve essere maggiore di $\mathcal{X}_1$ è il seguente:

$$\Rightarrow \langle (X_1, X_2), (3, 1), (3, 2), (2, 1) \rangle$$

$$\Rightarrow \langle (X_1, X_2), X_1 > X_2 \rangle$$

Nei CSP si effettuano **assegnamenti** di valori a variabili: $[\mathcal{X}_i = v_i, \mathcal{X}_j = v_j]$.
Un assegnamento che non viola nessun vincolo è chiamato **consistente** o **legale**.

assegnamento completo: se viene assegnato un valore a tutte le variabili $\Rightarrow$ **soluzione** di un problema è un *assegmanto completo* e *consistente*

assegnamento parziale: se alcune variabili rimangono non assegnate $\Rightarrow$ **soluzione parziale** di un problema è un *assegnamento parziale* e *consistente*

Risolvere un CSP è un problema NP-completo in generale, anche se esistono importanti sottoclassi di CSP che si possono risolvere in modo efficiente.

Esempio: colorazione di una mappa

Consideriamo la mappa dell'Australia.

Figura 5.1: mappa Australia da colorare

L'obiettivo è quello di colorare ogni regione di rosso, verde o blu, in modo tale che non ci siano regioni adiacenti con lo stesso colore.
Per formulare questo come CSP, definiamo alcune cose:

variabili : rappresentate da ogni regione australiana

$$\mathcal{X} = WA, NT, Q, NSW, V, SA, T$$

dominio : ogni variabile possiede il seguente dominio

$$\mathcal{D} = rosso, verde, blu$$

vincoli : le regioni adiacenti non possono avere lo stesso colore

$$\begin{aligned}\mathcal{C} = \{&(SA \neq WA), (SA \neq NT), (SA \neq Q), (SA \neq NSW),\\ &(SA \neq V), (WA \neq NT), (NT \neq Q), (Q \neq NSW),\\ &(NSW \neq V)\}\end{aligned}$$

Ci sono molte soluzioni possibili come:

$$\mathcal{C} = \{WA = \text{rosso},\ NT = \text{verde},\ Q = \text{rosso},\\ NSW = \text{verde},\ V = \text{rosso},\ SA = \text{blu},\ T = \text{rosso}\}$$

Figura 5.2: mappa Australia colorata

Può essere utile visualizzare un CSP come un **grafo di vincoli**: i nodi del grafo corripsondono alle variabili del problema e un arco connette ogni coppia di variabili che partecipano al vincolo.

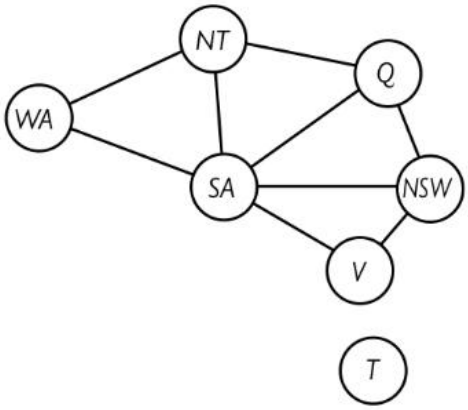


Figura 5.3: grafo della mappa dell'Australia

Prchè esprimere un problema sotto forma di CSP?
⇒ CSP costituiscono una rappresentazione naturale per un'ampia varietà di problemi
⇒ portano rapidamente grandi porzioni dello spazio degli stati, cosa che un algoritmo di ricerca non riesce a fare

5.1.1 Varianti del formalismo CSP

5.2 Programmazione dei vincoli: inferenza nei CSP

5.2.1 Consistenza di nodo

5.2.2 Consistenza d'arco

5.2.3 Consistenza di cammino

5.2.4 K-consistenza

5.2.5 Vincoli globali

5.3 Ricerca con backtracking per CSP